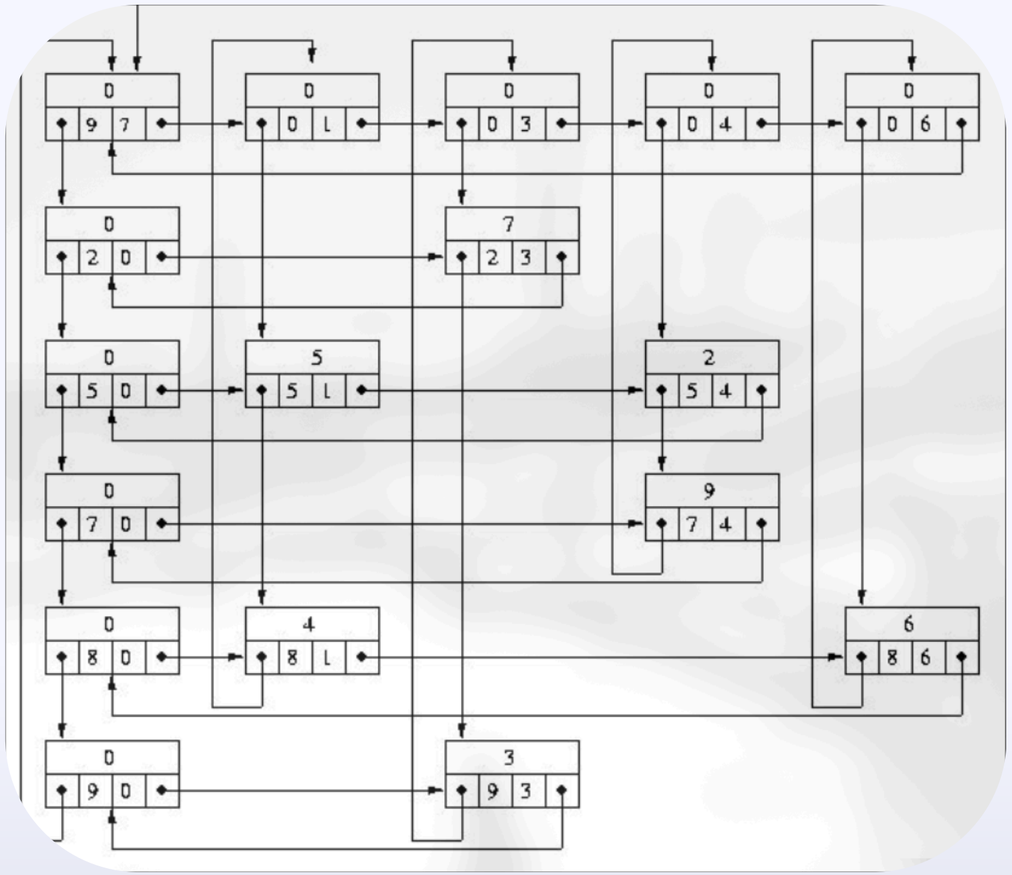
CS2100 - Algoritmos y Estructuras de Datos

Presentado por:

- JOSÉ LUIS ALVA ESPINOZA
- VALERIA LOCK GADEA
- AARON ADRIANO ROMANO CASTRO



IMPLEMENTACIÓN DE UNA HOJA DE CÁLCULO SIMPLE CON MATRICES DISPERSAS

MOTIVACIÓN

En una hoja de cálculo existen miles de celdas posibles, pero la mayoría están vacías.

PROBLEMA

- Una matriz densa reserva memoria para todas las celdas, incluso si están vacías.

Gran desperdicio de memoria.

SOLUCIÓN ADOPTADA

- Matriz dispersa con listas enlazadas cruzadas.
- Solo existen nodos para las celdas con contenido.

Mucho más eficiente en memoria y tiempo.

COMPARACIÓN CUANTITATIVA

- Ejemplo concreto: matriz de 1000 × 1000 con solo 1% de celdas ocupadas (10 000 celdas).
- **Matriz densa: ~8 MB** (suponiendo double).
  - Double (punto flotante de doble precisión) ocupa 8 bytes.
  - Total de posiciones: 1000 × 1000 = 1 000 000 celdas.
  - Memoria total: 1 000 000 celdas × 8 bytes = 8 000 000 bytes ≈ 8 MB.
- **Con lista enlazada:** solo almacenamos los 10 000 nodos → **menos de 1 MB.**
  - Cada nodo almacena: fila (int, 4 bytes) + columna (int, 4 bytes) + valor (double, 8 bytes) + puntero fila (8 bytes) + puntero columna (8 bytes) = 32 bytes por nodo (aprox.).
  - Memoria total: 10 000 nodos × 32 bytes ≈ 320 000 bytes ≈ 0.3 MB (menos de 1 MB).

VENTAJA ADICIONAL

- Inserciones y eliminaciones más rápidas porque no hay que reorganizar toda la estructura.

DISEÑO DE LA ESTRUCTURA

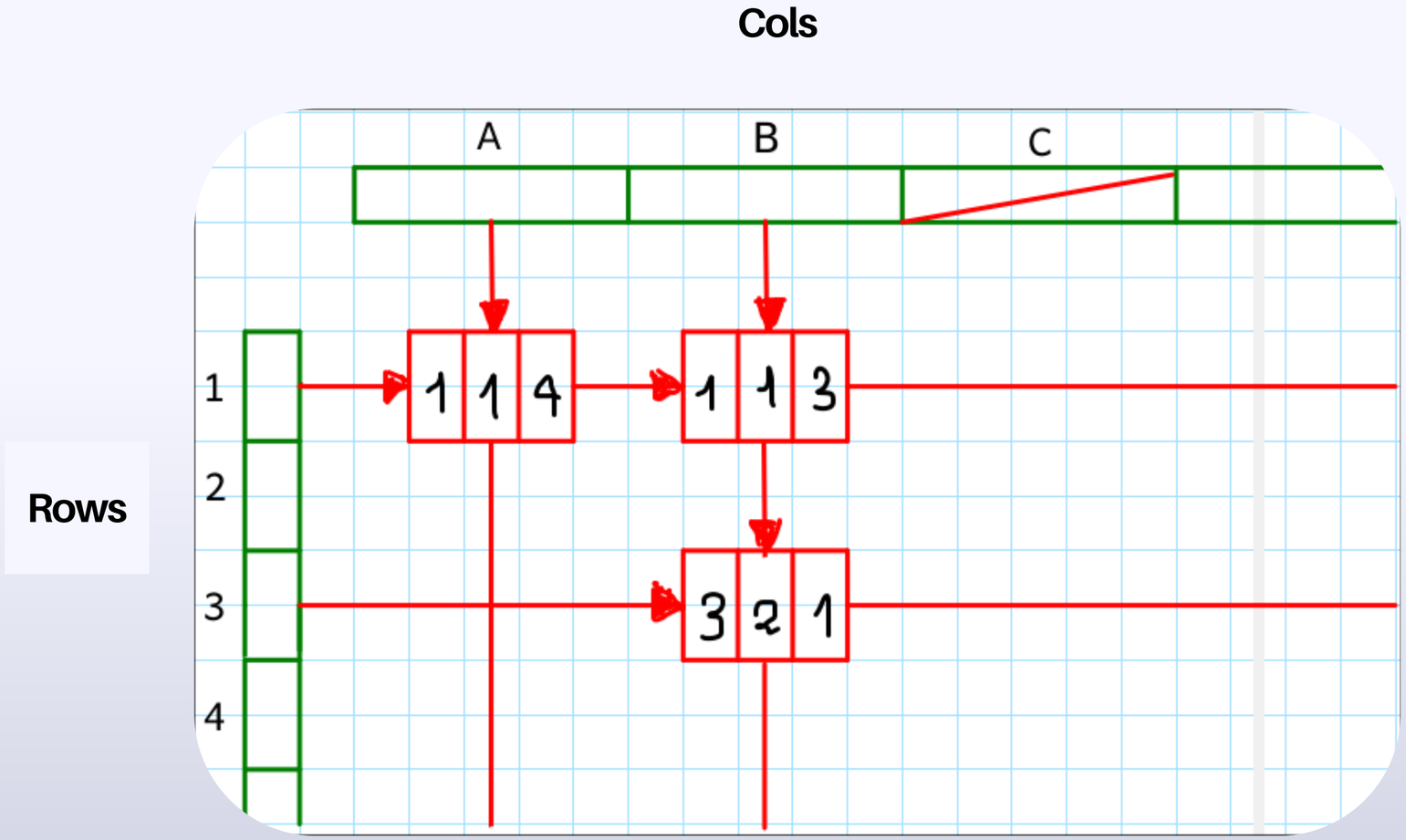
```
struct Node {
    string valor;
    int pos_row;
    int pos_col;
    Node *next_row; // siguiente en la misma fila (derecha)
    Node *next_col; // siguiente en la misma columna (abajo)

    Node(int r, int c, const string &v);
};

vector<Node *> rows; // crecerá dinámicamente
vector<Node *> cols;
```

DISEÑO DE LA ESTRUCTURA

Lista enlazada cruzada



ANÁLISIS DE LA COMPLEJIDAD

- *k_row*: celdas no vacías en la fila objetivo.
- *k_col*: celdas no vacías en la columna objetivo.
- *F*: filas activas totales.
- *C*: columnas activas totales.
- *n*: longitud de la fórmula.

`minRango`	O(F + k_total)	Determina el valor numérico mínimo en el rango.
`FormulaEvaluator::evaluar`	O(n)	Parser de precedencia que procesa la expresión linealmente.

Operación	Complejidad (peor caso)	Explicación
`insertarCelda`	O(F + C)	Redimensiona vectores y busca posición en fila y columna.
`consultarCelda`	O(k_row)	Accede por índice a la fila y recorre sus nodos.
`modificarCelda`	O(k_row)	Ídem a `consultarCelda`.
`eliminarCelda`	O(k_row + k_col)	Desenlaza el nodo de su fila y de su columna.
`eliminarFila`	O(k_row × C)	Desenlaza cada nodo de su respectiva columna.
`eliminarColumna`	O(k_col × F)	Análogo a `eliminarFila`.
`eliminarRango`	O((f2−f1)×(c2−c1))	Invoca `eliminarCelda` por cada celda del rectángulo.
`sumaFila`	O(k_row)	Recorre los nodos de la fila y acumula valores numéricos.
`sumaColumna`	O(k_col)	Recorre los nodos de la columna.
`sumaRango`	O(F + k_total)	Itera sobre las filas del rango y recorre los nodos presentes.
`promedioFila`	O(k_row)	Cuenta y suma los valores numéricos de la fila.
`promedioColumna`	O(k_col)	Análogo sobre una columna.
`promedioRango`	O(F + k_total)	Recorre el rectángulo contando y sumando.
`maxRango`	O(F + k_total)	Determina el valor numérico máximo en el rango.

IMPLEMENTACIÓN CLAVE

- Inserción y eliminación de una celda

```cpp
void SparseMatrix::insertarCelda(int f, int c, const string &valor) {
  if (f < 0 || c < 0) return;
  if (valor.empty()) return;

  if (static_cast<size_t>(f) ≥ rows.size()) rows.resize(new_size: f + 1, x: nullptr);
  if (static_cast<size_t>(c) ≥ cols.size()) cols.resize(new_size: c + 1, x: nullptr);

  Node *curr = rows[f];
  while (curr && curr->pos_col ≤ c) {
    if (curr->pos_col == c) {
      curr->valor = valor;
      return;
    }
    curr = curr->next_col;
  }

  Node *nuevo = new Node(r: f, c, v: valor);

  Node *prevCol = nullptr;
  curr = rows[f];
  while (curr && curr->pos_col < c) {
    prevCol = curr;
    curr = curr->next_col;
  }
  nuevo->next_col = curr;
  if (!prevCol) rows[f] = nuevo;
  else  prevCol->next_col = nuevo;

  Node *prevRow = nullptr;
  curr = cols[c];
  while (curr && curr->pos_row < f) {
    prevRow = curr;
    curr = curr->next_row;
  }
  nuevo->next_row = curr;
  if (!prevRow) cols[c] = nuevo;
  else prevRow->next_row = nuevo;
}
```

```cpp
bool SparseMatrix::eliminarCelda(int f, int c) {
  if (f < 0 || c < 0) return false;
  if (static_cast<size_t>(f) ≥ rows.size() ||
      static_cast<size_t>(c) ≥ cols.size())
    return false;

  Node *prev = nullptr;
  Node *curr = rows[f];
  while (curr && curr->pos_col < c) {
    prev = curr;
    curr = curr->next_col;
  }
  if (!curr || curr->pos_col ≠ c) return false;

  if (prev) prev->next_col = curr->next_col;
  else rows[f] = curr->next_col;

  Node *prevRow = nullptr;
  Node *currCol = cols[c];
  while (currCol && currCol->pos_row < f) {
    prevRow = currCol;
    currCol = currCol->next_row;
  }
  if (currCol && currCol->pos_row == f) {
    if (prevRow) prevRow->next_row = currCol->next_row;
    else cols[c] = currCol->next_row;
  }

  delete curr;
  return true;
}
```

IMPLEMENTACIÓN CLAVE

- Eliminación de fila y columna

```cpp
void SparseMatrix::eliminarFila(int f) {
    if (f < 0 || static_cast<size_t>(f) ≥ rows.size() || !rows[f])
        return;
    Node *curr = rows[f];
    while (curr) {
        int c = curr->pos_col;
        Node *next = curr->next_col;
        if (static_cast<size_t>(c) < cols.size() && cols[c]) {
            Node *prev = nullptr;
            Node *colCurr = cols[c];
            while (colCurr && colCurr->pos_row < f) {
                prev = colCurr;
                colCurr = colCurr->next_row;
            }
            if (colCurr && colCurr->pos_row == f) {
                if (prev)
                    prev->next_row = colCurr->next_row;
                else
                    cols[c] = colCurr->next_row;
            }
        }
        delete curr;
        curr = next;
    }
    rows[f] = nullptr;
}
```

```cpp
void SparseMatrix::eliminarColumna(int c) {
    if (c < 0 || static_cast<size_t>(c) ≥ cols.size() || !cols[c])
        return;
    Node *curr = cols[c];
    while (curr) {
        int f = curr->pos_row;
        Node *next = curr->next_row;
        if (static_cast<size_t>(f) < rows.size() && rows[f]) {
            Node *prev = nullptr;
            Node *rowCurr = rows[f];
            while (rowCurr && rowCurr->pos_col < c) {
                prev = rowCurr;
                rowCurr = rowCurr->next_col;
            }
            if (rowCurr && rowCurr->pos_col == c) {
                if (prev)
                    prev->next_col = rowCurr->next_col;
                else
                    rows[f] = rowCurr->next_col;
            }
        }
        delete curr;
        curr = next;
    }
    cols[c] = nullptr;
}
```

EN RESUMEN:

Inserción

El nodo nuevo apunta al siguiente en su fila y al siguiente en su columna. El nodo anterior de cada lista actualiza su puntero para apuntar al nuevo.

Eliminación de celda

El nodo anterior apunta directamente al siguiente, saltándose el nodo eliminado. Esto se hace en ambas listas antes de llamar a delete

Eliminación de fila

Por cada nodo de la fila, se actualiza el puntero de su columna para saltárselo, luego se hace delete.
Al final rows[f] = nullptr."

Eliminación de columna

Mismo proceso que en fila, pero recorriendo la columna y actualizando los punteros de cada fila afectada.

Al final cols[c] = nullptr

FORMULAEVALUATOR

```
> FormulaEvaluator::FormulaEvaluator(CellProvider* matrix) {···

> void FormulaEvaluator::skipBlank() {···

> double FormulaEvaluator::evaluate(const string& inputFormula) {···

> double FormulaEvaluator::evalExpression() {···

> double FormulaEvaluator::evalTerm() {···

> double FormulaEvaluator::evalFactor() {···

> double FormulaEvaluator::readNumber() {···

> double FormulaEvaluator::readCell() {···

> pair<int, int> FormulaEvaluator::cellRefToIndex(const string& cellName) {···
```

FORMULAEVALUATOR

```
FormulaEvaluator::FormulaEvaluator(CellProvider* matrix) {
    this->matrix = matrix;
    index = 0;
}

void FormulaEvaluator::skipBlank() {
    while (index < (int)formula.size() && isspace(formula[index])) {
        index++;
    }
}
```

FormulaEvaluator

Recibe un puntero a CellProvider y lo guarda para poder leer valores de otras celdas. También inicializa index en 0, que es el cursor que recorre la fórmula.

skipBlank

Avanza index mientras haya espacios en blanco. Se usa principalemente para ignorar separadores.

FORMULAEVALUATOR

```cpp
double FormulaEvaluator::evaluate(const string& inputFormula) {
    if (inputFormula.empty() || inputFormula[0] != '=') {
        throw runtime_error("La formula debe empezar con =");
    }

    formula = inputFormula.substr(1);
    index = 0;

    double answer = evalExpression();

    skipBlank();

    if (index != (int)formula.size()) {
        throw runtime_error("Formula invalida");
    }

    return answer;
}
```

evaluate

Verifica que la fórmula empiece con =, evalúa toda la expresión y luego comprueba que no hayan quedado caracteres sin consumir.

```cpp
double FormulaEvaluator::evalExpression() {
    double leftValue = evalTerm();

    while (true) {
        skipBlank();

        if (index >= (int)formula.size()) break;

        char symbol = formula[index];

        if (symbol != '+' && symbol != '-') break;

        index++;

        double nextValue = evalTerm();

        if (symbol == '+') {
            leftValue += nextValue;
        } else {
            leftValue -= nextValue;
        }
    }

    return leftValue;
}
```

evalExpression

Evalúa sumas y restas

FORMULAEVALUATOR

```cpp
double FormulaEvaluator::evalTerm() {
    double leftValue = evalFactor();

    while (true) {
        skipBlank();

        if (index >= (int)formula.size()) break;

        char symbol = formula[index];

        if (symbol != '*' && symbol != '/') break;

        index++;

        double nextValue = evalFactor();

        if (symbol == '*') {
            leftValue *= nextValue;
        } else {
            if (nextValue == 0) {
                throw runtime_error("Division por cero");
            }
            leftValue /= nextValue;
        }
    }

    return leftValue;
}
```

evalTerm

Evalúa multiplicaciones y divisiones

```cpp
double FormulaEvaluator::evalFactor() {
    skipBlank();

    if (index >= (int)formula.size()) {
        throw runtime_error("Formula incompleta");
    }

    if (formula[index] == '(') {
        index++;

        double inside = evalExpression();

        skipBlank();

        if (index >= (int)formula.size() || formula[index] != ')') {
            throw runtime_error("Falta cerrar parentesis");
        }

        index++;
        return inside;
    }

    if (isdigit(formula[index]) || formula[index] == '.' || formula[index] == '-') {
        return readNumber();
    }

    if (isalpha(formula[index])) {
        return readCell();
    }

    throw runtime_error("Caracter invalido en formula");
}
```

evalFactor

Resuelve una unidad básica de la expresión (expresión entre paréntesis, un número o referencia de celda)

FORMULAEVALUATOR

```cpp
double FormulaEvaluator::readNumber() {
    skipBlank();

    int beginPos = index;

    if (formula[index] == '-') {
        index++;
    }

    bool foundDigit = false;
    bool foundPoint = false;

    while (index < (int)formula.size()) {
        if (isdigit(formula[index])) {
            foundDigit = true;
            index++;
        } else if (formula[index] == '.') {
            if (foundPoint) break;
            foundPoint = true;
            index++;
        } else {
            break;
        }
    }

    if (!foundDigit) {
        throw runtime_error("Numero invalido");
    }

    return stod(formula.substr(beginPos, index - beginPos));
}
```

readNumber

Lee un número textual desde la fórmula. Acepta signo negativo, enteros y decimales con punto.

```cpp
double FormulaEvaluator::readCell() {
    skipBlank();
    int beginPos = index;

    while (index < (int)formula.size() && isalpha(formula[index])) {
        index++;
    }

    while (index < (int)formula.size() && isdigit(formula[index])) {
        index++;
    }

    string cellName = formula.substr(beginPos, index - beginPos);
    pair<int, int> coords = cellRefToIndex(cellName);
    int row = coords.first;
    int col = coords.second;
    string cellValue = matrix->getCellValue(row, col);

    if (cellValue.empty()) {
        throw runtime_error("Referencia vacia: " + cellName);
    }

    try {
        size_t charsUsed = 0;
        double numericValue = stod(cellValue, &charsUsed);

        if (charsUsed != cellValue.size()) {
            throw runtime_error("Celda no numerica: " + cellName);
        }

        return numericValue;
    } catch (...) {
        throw runtime_error("Celda no numerica: " + cellName);
    }
}
```

readCell

Lee una referencia de celda y lo convierte a número

FORMULAEVALUATOR

```cpp
pair<int, int> FormulaEvaluator::cellRefToIndex(const string& cellName) {
    int ptr = 0;
    int columnIndex = 0;

    while (ptr < (int)cellName.size() && isalpha(cellName[ptr])) {
        columnIndex = columnIndex * 26 + (toupper(cellName[ptr]) - 'A' + 1);
        ptr++;
    }

    if (columnIndex == 0) {
        throw runtime_error("Referencia invalida: " + cellName);
    }

    int rowIndex = 0;

    while (ptr < (int)cellName.size() && isdigit(cellName[ptr])) {
        rowIndex = rowIndex * 10 + (cellName[ptr] - '0');
        ptr++;
    }

    if (rowIndex == 0 || ptr != (int)cellName.size()) {
        throw runtime_error("Referencia invalida: " + cellName);
    }

    return {rowIndex - 1, columnIndex - 1};
}
```

cellRefToIndex

Convierte una referencia en índices. Las letras se interpretan como columna en base 26 y los números como fila.

Por ejemplo, A1 pasa a {1, 1}

EN RESUMEN:

Cadena de uso

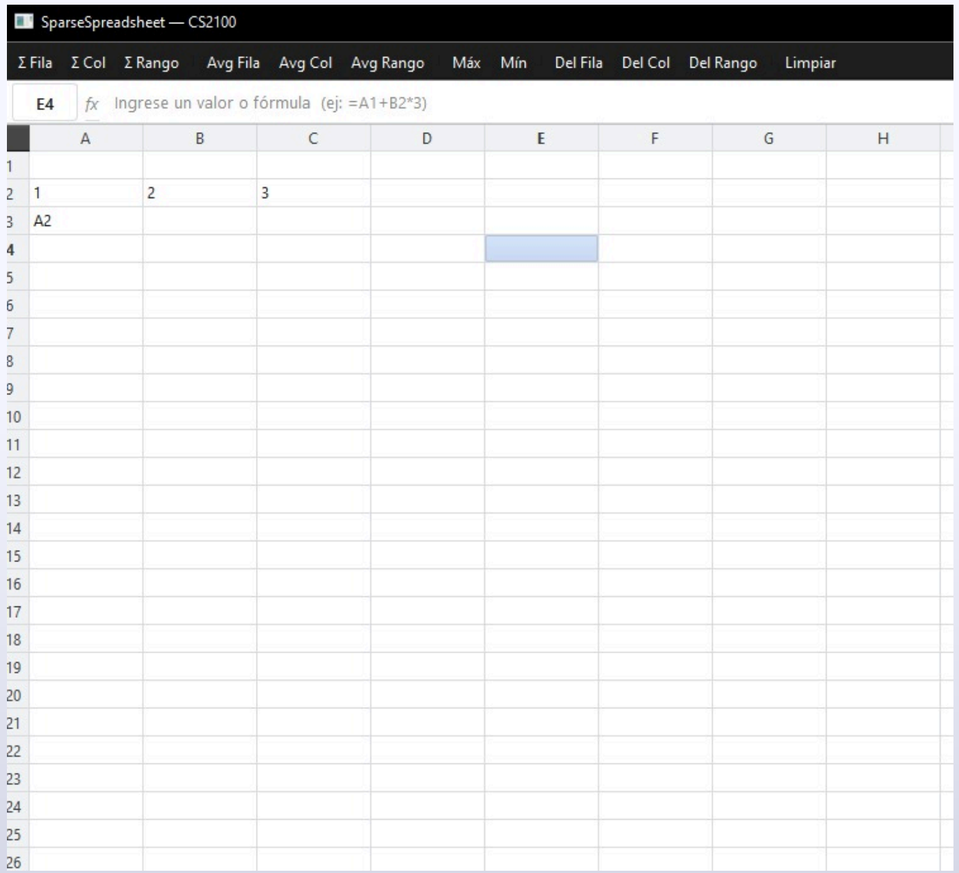
1. evaluate
2. evalExpression
3. evalTerm
4. evalFactor
    a. Si hay un parentesis:
        2. evalExpression

Ejemplo: =2*(A1+3)

1. Se elimina el = y empieza el análisis. // evaluate
2. Se lee el primer término. // evalExpression
3. Se lee el factor 2 //evalTerm
4. Luego encuentra el operador * y vuelve a pedir otro factor.
    a. Hay un parentesis abierto, llama a evalExpression
    b. Se procesa procesa A1+3
    c. Cuando encuentra el), regresa el resultado al nivel anterior

Existe recursión si la expresión contiene otra expresión (Si la fórmula tiene paréntesis)

INTERFAZ QT



- Tabla editable
- Barra de fórmulas
- Botones: Suma, Promedio, Max/Min, Eliminar fila/columna, Eliminar rango

DEMO EN VIVO

SECUENCIA DE ACCIONES:

INSERTAR: A1=10, B1=20, B2=2, A2=TEXTO

CONSULTAR A1 → 10

AGREGACIÓN: SUMAFILA(1) → 30, MAXRANGO(A1:B2) → 20

ELIMINAR FILA 1 Y VERIFICAR QUE A1 AHORA ESTÁ VACÍO

FÓRMULAS: =A1+B1 → 30; =A1/B2 → 5; =A1/B2 CON B2 BORRADO → ERROR "REFERENCIA VACÍA"

CASO BORDE: ELIMINAR UNA FILA YA VACÍA NO PRODUCE ERROR; PROMEDIO EN RANGO SIN NÚMEROS → 0.0

GRACIAS POR SU ATENCIÓN